

**LAPORAN PROJECT AKHIR STRUKTUR DATA
SISTEM REKOMENDASI HANDPHONE**



Dosen Pengampu:

Yulya Muharmi, M.Kom.

Disusun oleh Kelompok 9 :

- | | |
|--------------------------|------------|
| 1.Dana Risqi Wijaya | 2517052063 |
| 2.Pandu Winata | 2557052010 |
| 3. Muhammad Hibatullah | 2517052052 |
| 4.Sekar Ayu Maheswari | 2557052006 |
| 5.Gita Tri Juliet Marbun | 2517052035 |

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS LAMPUNG**

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan industri teknologi, khususnya pada perangkat gawai pintar (handphone), mengalami peningkatan yang sangat pesat dalam beberapa tahun terakhir. Berbagai produsen berlomba-lomba menghadirkan produk dengan spesifikasi yang beragam, seperti kapasitas RAM, penyimpanan internal, harga, hingga rating kepuasan pengguna. Banyaknya variasi pilihan ini memang memberikan keleluasaan bagi konsumen, namun di sisi lain juga menimbulkan permasalahan baru berupa kesulitan dalam menentukan pilihan yang paling sesuai. Kondisi ini dikenal sebagai *choice overload*, di mana pengguna membutuhkan waktu lebih lama untuk membandingkan dan memilih produk yang sesuai dengan kebutuhan serta anggaran yang dimiliki.

Untuk mengatasi permasalahan tersebut, diperlukan sebuah sistem berbasis komputer yang mampu mengelola data handphone secara efisien sekaligus memberikan rekomendasi secara cepat, tepat, dan interaktif. Efektivitas sistem ini sangat dipengaruhi oleh bagaimana data disusun serta bagaimana algoritma digunakan untuk mengolah data tersebut. Oleh karena itu, penerapan struktur data dan algoritma menjadi aspek penting dalam pengembangan sistem ini. Penggunaan struktur data seperti list dinamis memungkinkan penyimpanan data yang fleksibel, sementara algoritma seperti linear search dan selection sort digunakan untuk proses pencarian serta pengurutan data secara efisien.

Selain itu, untuk meningkatkan kualitas sistem dan pengalaman pengguna, diperlukan fitur tambahan yang mendukung interaksi yang lebih baik. Misalnya, fitur perbandingan produk yang memanfaatkan struktur data antrean (queue) dengan prinsip FIFO untuk membatasi jumlah data yang dibandingkan. Selanjutnya, pencatatan riwayat aktivitas pengguna dapat dilakukan menggunakan Double Linked List agar navigasi maju dan mundur dapat dilakukan dengan cepat. Tidak hanya itu, penggunaan Binary Search Tree (BST) juga diterapkan untuk mempercepat proses pencarian data berdasarkan nilai rating tanpa harus memeriksa seluruh data.

Berdasarkan hal tersebut, pada proyek ini dikembangkan sebuah aplikasi sistem rekomendasi handphone berbasis desktop menggunakan bahasa pemrograman Python dengan antarmuka grafis Tkinter. Aplikasi ini mengintegrasikan berbagai konsep penting, seperti Pemrograman Berorientasi Objek (OOP), list dinamis, Double Linked List, queue FIFO, Binary Search Tree, linear search, dan selection sort. Dengan pendekatan ini, diharapkan sistem tidak hanya membantu pengguna dalam mengambil keputusan, tetapi juga menunjukkan bagaimana penerapan struktur data dan algoritma dapat meningkatkan efisiensi kinerja program.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana menerapkan konsep pemrograman berorientasi objek untuk merepresentasikan data handphone dengan berbagai atribut?
2. Bagaimana merancang struktur data list dinamis untuk mendukung pengelolaan data secara real-time?
3. Bagaimana mengimplementasikan Double Linked List sebagai media pencatatan riwayat aktivitas pengguna?
4. Bagaimana menerapkan struktur antrean (queue) berbasis FIFO untuk membatasi data perbandingan produk?
5. Bagaimana membangun Binary Search Tree untuk mempercepat proses pencarian data berdasarkan rating?
6. Bagaimana menggabungkan algoritma linear search dan selection sort dalam menghasilkan rekomendasi produk?
7. Bagaimana merancang antarmuka grafis menggunakan Tkinter yang mampu menampilkan data secara interaktif dan responsif?

1.3 Tujuan Penelitian

Adapun tujuan dari penelitian dan pengembangan sistem ini adalah sebagai berikut:

1. Mengembangkan kelas objek handphone dengan konsep enkapsulasi untuk menyatukan berbagai atribut dalam satu entitas.
2. Menyediakan media penyimpanan data menggunakan list dinamis yang fleksibel untuk operasi tambah, ubah, dan hapus data.
3. Membangun sistem riwayat aktivitas menggunakan Double Linked List yang memungkinkan navigasi cepat ke data sebelumnya dan berikutnya.
4. Menerapkan antrean FIFO untuk mengatur batas maksimal perbandingan produk.
5. Mengoptimalkan pencarian data berdasarkan rating menggunakan Binary Search Tree dengan kompleksitas lebih efisien.
6. Menghasilkan sistem rekomendasi yang mampu menyaring dan mengurutkan produk sesuai dengan kebutuhan dan anggaran pengguna.
7. Mengembangkan aplikasi desktop yang interaktif, mudah digunakan, serta memiliki sinkronisasi data yang baik antara tampilan dan penyimpanan data.

BAB II

IMPLEMENTASI STRUKTUR DATA

2.1 Pemrograman Berorientasi Objek (Object-Oriented Programming)

Berikut adalah implementasi pembuatan kelas entitas handphone menggunakan prinsip dasar pemrograman berorientasi objek:

```
4 class Handphone:
5     def __init__(self, id_hp, nama, harga, ram, storage, rating):
6         self.id = id_hp
7         self.nama = nama
8         self.harga = harga
9         self.ram = ram
10        self.storage = storage
11        self.rating = rating
12
13    def __str__(self):
14        return f"{self.nama} (RAM: {self.ram}GB) - Rp{self.harga:,} [★{self.rating}]"
15
```

Penjelasan:

Penerapan konsep pemrograman berorientasi objek pada kode program tersebut direalisasikan melalui pendefinisian sebuah kelas independen bernama *Handphone*. Kelas ini berfungsi sebagai kerangka dasar (blueprint) yang menggambarkan karakteristik umum dari setiap perangkat gawai yang dikelola dalam sistem. Di dalamnya terdapat metode inisialisasi yang berperan sebagai konstruktor, yang akan otomatis dijalankan saat objek baru dibuat. Konstruktor ini memiliki enam atribut utama, yaitu *self.id*, *self.nama*, *self.harga*, *self.ram*, *self.storage*, dan *self.rating*. Setiap parameter yang diterima akan langsung disimpan ke dalam atribut tersebut dengan bantuan kata kunci *self* sebagai penunjuk referensi objek.

Penggunaan pendekatan berorientasi objek ini sangat penting dalam pengembangan perangkat lunak karena mengimplementasikan prinsip enkapsulasi. Prinsip ini memungkinkan berbagai data dengan tipe yang berbeda—seperti teks (*string*) untuk nama, bilangan bulat (*integer*) untuk harga, RAM, dan penyimpanan, serta bilangan desimal (*float*) untuk rating—digabungkan menjadi satu kesatuan objek yang utuh. Dengan demikian, setiap data gawai diperlakukan sebagai entitas mandiri, sehingga meminimalkan risiko kesalahan manipulasi data dan meningkatkan keamanan serta kerapihan struktur program.

Selain itu, kelas ini juga dilengkapi dengan metode khusus yang digunakan untuk merepresentasikan objek dalam bentuk teks. Metode ini secara otomatis menentukan bagaimana objek akan ditampilkan saat dicetak ke layar. Proses ini melibatkan

penyusunan teks yang mencakup nama perangkat, kapasitas RAM dengan satuan gigabyte, format harga dengan pemisah ribuan, serta nilai rating yang ditampilkan dengan simbol bintang. Adanya metode ini membuat proses penampilan data menjadi lebih efisien karena tidak perlu lagi mengambil atribut satu per satu secara manual, sehingga mengurangi redundansi kode pada bagian antarmuka pengguna.

2.2 Struktur Data Array / Koleksi Dinamis (List Collection)

Berikut adalah implementasi alokasi penyimpanan basis data internal menggunakan struktur data koleksi list dinamis:

```
~~~
137         self.database_hp = [
138             Handphone(1, "Infinix Note 40", 2500000, 8, 256, 4.5),
139             Handphone(2, "Samsung Galaxy A55", 5900000, 8, 256, 4.7),
140             Handphone(3, "Xiaomi Redmi Note 13", 2300000, 6, 128, 4.3),
141             Handphone(4, "iPhone 15 Pro", 19000000, 8, 128, 4.9),
142             Handphone(5, "Vivo V30", 4200000, 12, 512, 4.6)
143         ]
144         self.id_counter = 6
```

Penjelasan

Penyimpanan data utama dalam program ini menggunakan struktur data list Python yang diberi nama *database_hp*, yang dideklarasikan sebagai variabel instans di dalam konstruktor kelas utama antarmuka aplikasi. List tersebut berfungsi sebagai wadah dinamis yang menyimpan kumpulan objek yang dihasilkan dari kelas *Handphone*. Secara konsep, list ini bertindak sebagai array sekuensial yang tersimpan di memori RAM dan digunakan untuk menampung data gawai secara berurutan.

Saat program dijalankan pertama kali, list ini telah diinisialisasi dengan lima objek awal yang dimasukkan secara statis. Masing-masing objek berisi informasi lengkap seperti ID unik (dimulai dari 1 hingga 5), nama produk, harga, kapasitas RAM, penyimpanan internal, serta nilai rating dalam bentuk desimal. Selain itu, sistem juga menyiapkan variabel tambahan bernama *id_counter* yang diinisialisasi dengan nilai awal 6 sebagai penunjang proses penomoran data berikutnya.

Keunggulan penggunaan list dinamis terletak pada fleksibilitasnya dalam pengelolaan memori. Berbeda dengan array statis yang memiliki ukuran tetap sejak awal, list dapat secara otomatis menyesuaikan kapasitasnya selama program berjalan. Ketika pengguna menambahkan data baru, sistem akan memanfaatkan metode *append* untuk memasukkan objek *Handphone* ke bagian akhir list, sehingga panjang list bertambah secara otomatis tanpa perlu pengaturan ukuran manual.

Sebaliknya, saat proses penghapusan data dilakukan, sistem akan membentuk ulang list menggunakan teknik *list comprehension* dengan cara menyaring elemen berdasarkan ID tertentu. Elemen yang sesuai dengan kriteria penghapusan akan dihilangkan, sehingga

ukuran list akan berkurang secara dinamis. Dalam hal ini, variabel *id_counter* memiliki peran penting sebagai penghasil ID otomatis. Setiap penambahan data baru akan menggunakan nilai dari *id_counter* sebagai identitas unik, kemudian nilainya akan ditingkatkan secara bertahap. Mekanisme ini memastikan bahwa setiap objek dalam list memiliki ID yang unik, terurut, dan tidak saling bertabrakan, sehingga memudahkan proses pengelolaan dan pencarian data dalam sistem.

2.3 Struktur Data Double Linked List (Manual)

Berikut adalah implementasi struktur data rantai tautan ganda yang dibangun secara manual tanpa bantuan pustaka eksternal untuk mengelola log navigasi:

```
45
16     class DLLNode:
17         def __init__(self, data):
18             self.data = data
19             self.next = None
20             self.prev = None
21
22     class HistoryLinkedList:
23         def __init__(self):
24             self.head = None
25             self.tail = None
26             self.current = None
27
28         def add_history(self, hp_object):
29             new_node = DLLNode(hp_object)
30             if not self.head:
31                 self.head = new_node
32                 self.tail = new_node
33             else:
34                 self.tail.next = new_node
35                 new_node.prev = self.tail
36                 self.tail = new_node
37                 self.current = self.tail
38
39         def get_previous(self):
40             if self.current and self.current.prev:
41                 self.current = self.current.prev
42             return self.current.data
43         return None
44
45         def get_next(self):
46             if self.current and self.current.next:
47                 self.current = self.current.next
48             return self.current.data
49         return None
50
```

Penjelasan:

Struktur data non-array linier berupa Double Linked List pada program ini diimplementasikan melalui dua kelas utama, yaitu *DLLNode* dan *HistoryLinkedList*. Kelas *DLLNode* berfungsi sebagai representasi satu simpul (node) dalam memori. Pada

konstruktor kelas ini, terdapat atribut *self.data* yang menyimpan objek handphone, serta dua atribut pointer, yaitu *self.next* dan *self.prev*, yang masing-masing digunakan untuk menunjuk ke simpul berikutnya dan sebelumnya. Kedua pointer tersebut diinisialisasi dengan nilai kosong sebagai penanda bahwa belum terhubung dengan simpul lain.

Sementara itu, kelas *HistoryLinkedList* bertindak sebagai pengelola struktur Double Linked List secara keseluruhan, khususnya untuk menyimpan riwayat interaksi atau eksplorasi produk oleh pengguna. Pada saat inisialisasi, kelas ini menetapkan tiga pointer utama, yaitu *self.head* sebagai penunjuk simpul pertama, *self.tail* sebagai penunjuk simpul terakhir, dan *self.current* sebagai penanda posisi simpul yang sedang aktif diakses.

Proses penambahan data riwayat diatur dalam metode *add_history*. Ketika metode ini menerima objek handphone, objek tersebut terlebih dahulu dibungkus ke dalam node baru menggunakan kelas *DLLNode*. Jika struktur masih kosong (ditandai dengan *self.head* bernilai kosong), maka node baru langsung dijadikan sebagai node pertama sekaligus terakhir, dengan *self.head* dan *self.tail* menunjuk ke node tersebut. Namun, jika data sudah ada sebelumnya, maka node baru akan ditambahkan di bagian akhir. Proses ini dilakukan dengan menghubungkan node terakhir lama ke node baru melalui pointer *next*, serta menghubungkan node baru ke node sebelumnya melalui pointer *prev*. Setelah itu, pointer *self.tail* diperbarui agar menunjuk ke node terbaru. Terakhir, pointer *self.current* juga diperbarui agar selalu berada pada node paling akhir sebagai posisi aktif.

Navigasi dalam struktur ini dilakukan melalui metode *get_previous* dan *get_next*. Pada metode *get_previous*, sistem akan memeriksa apakah node saat ini memiliki node sebelumnya. Jika ada, maka pointer *self.current* digeser ke node sebelumnya melalui *self.current.prev*, kemudian data pada node tersebut dikembalikan. Sebaliknya, metode *get_next* akan memeriksa keberadaan node berikutnya melalui *self.current.next*. Jika tersedia, pointer akan digeser maju ke node tersebut dan datanya dikembalikan.

Penerapan Double Linked List ini memberikan efisiensi tinggi dalam proses navigasi data, karena perpindahan antar node dapat dilakukan secara langsung tanpa perlu melakukan pencarian ulang dari awal. Kompleksitas waktu untuk operasi ini adalah $O(1)$, karena hanya melibatkan perpindahan pointer ke node yang sudah terhubung secara langsung.

2.4 Implementasi Struktur Data Queue

Berikut adalah implementasi struktur data antrean linier berbasis batasan ukuran untuk mengakomodasi fitur perbandingan spesifikasi produk:

```

class ComparisonQueue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        if len(self.items) >= 3:
            return False
        self.items.append(item)
        return True

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        return None

    def is_empty(self):
        return len(self.items) == 0

    def get_all(self):
        return self.items

```

Struktur data antrean pada program ini diimplementasikan melalui sebuah kelas khusus bernama *ComparisonQueue*. Kelas ini memanfaatkan list internal *self.items* yang diatur sedemikian rupa sehingga mengikuti prinsip First In First Out (FIFO), yaitu elemen yang masuk terlebih dahulu akan keluar lebih dahulu. Antrean ini digunakan untuk mendukung fitur perbandingan produk handphone yang dipilih oleh pengguna. Pada konstruktor kelas, list penampung diinisialisasi dalam kondisi kosong. Selain itu, kelas ini menyediakan beberapa metode utama, yaitu *enqueue* untuk menambahkan data, *dequeue* untuk menghapus data, *is_empty* untuk memeriksa apakah antrean kosong, serta *get_all* untuk mengambil seluruh data yang ada di dalam antrean guna keperluan tampilan pada antarmuka.

Metode *enqueue* dirancang dengan menerapkan konsep antrean terbatas (*bounded queue*). Sebelum menambahkan elemen baru, sistem terlebih dahulu mengecek jumlah elemen dalam list menggunakan fungsi *len*. Jika jumlah elemen telah mencapai tiga atau lebih, maka proses penambahan akan dihentikan, fungsi mengembalikan nilai *False*, dan data baru tidak akan dimasukkan. Pembatasan ini bertujuan untuk menjaga kerapian tampilan serta mengontrol penggunaan memori. Namun, jika kapasitas masih tersedia, maka sistem akan menambahkan objek handphone ke bagian akhir antrean menggunakan metode *append*, dan mengembalikan nilai *True* sebagai tanda bahwa proses berhasil.

Sementara itu, metode *dequeue* berfungsi untuk mengeluarkan elemen yang berada di posisi paling depan antrian. Sebelum proses dilakukan, sistem akan memeriksa terlebih dahulu apakah antrian kosong melalui metode *is_empty* untuk menghindari kesalahan. Jika antrian tidak kosong, maka metode *pop(0)* akan dijalankan untuk menghapus elemen pertama. Proses ini mencerminkan prinsip FIFO, di mana elemen yang paling awal dimasukkan akan dikeluarkan terlebih dahulu. Selain itu, penghapusan elemen ini juga menyebabkan pergeseran posisi elemen lainnya ke depan secara otomatis. Data yang dikeluarkan kemudian dikembalikan untuk ditampilkan kepada pengguna sebagai informasi.

Dengan penerapan struktur antrian yang memiliki batas kapasitas ini, pengelolaan data perbandingan produk dapat berjalan secara teratur, efisien, dan sesuai urutan interaksi pengguna, sehingga memberikan pengalaman penggunaan yang lebih terstruktur dan mudah dipahami.

2.5 Struktur Data Pohon Pencarian Biner (Binary Search Tree)

Berikut adalah implementasi model hierarki pohon biner terurut untuk mengoptimalkan pelacakan data berdasarkan nilai rating:

```

71
72     class BSTNode:
73         def __init__(self, hp_object):
74             self.hp = hp_object
75             self.left = None
76             self.right = None
77
78     class HandphoneBST:
79         def __init__(self):
80             self.root = None
81
82         def insert(self, hp_object):
83             if not self.root:
84                 self.root = BSTNode(hp_object)
85             else:
86                 self._insert_recursive(self.root, hp_object)
87
88         def _insert_recursive(self, node, hp_object):
89             if hp_object.rating < node.hp.rating:
90                 if not node.left:
91                     node.left = BSTNode(hp_object)
92                 else:
93                     self._insert_recursive(node.left, hp_object)
94             else:
95                 if not node.right:
96                     node.right = BSTNode(hp_object)
97                 else:
98                     self._insert_recursive(node.right, hp_object)
99
100        def search_exact_rating(self, rating_target):
101            return self._search_recursive(self.root, rating_target)
102
103        def _search_recursive(self, node, rating_target):
104            if not node or node.hp.rating == rating_target:
105                return node.hp if node else None
106            if rating_target < node.hp.rating:
107                return self._search_recursive(node.left, rating_target)
108            return self._search_recursive(node.right, rating_target)

```

Penjelasan:

Struktur data non-linier berbentuk hierarki pada program ini diimplementasikan melalui pembangunan Binary Search Tree (BST) menggunakan dua kelas utama, yaitu *BSTNode* dan *HandphoneBST*. Kelas *BSTNode* berperan sebagai representasi satu simpul dalam pohon biner, dengan tiga atribut utama, yaitu *self.hp* yang menyimpan objek handphone, serta *self.left* dan *self.right* yang berfungsi sebagai pointer untuk menunjuk ke anak kiri dan anak kanan. Sementara itu, kelas *HandphoneBST* bertindak sebagai pengelola keseluruhan struktur pohon, dengan atribut *self.root* sebagai penunjuk simpul akar. Pada awal eksekusi program, pohon berada dalam kondisi kosong dengan nilai *self.root* yang belum terisi.

Proses penambahan data dilakukan melalui metode *insert* yang menggunakan pendekatan rekursif. Ketika sebuah objek handphone dimasukkan, sistem terlebih dahulu memeriksa apakah akar pohon masih kosong. Jika ya, maka objek tersebut langsung dijadikan sebagai akar dengan membungkusnya ke dalam node baru. Namun,

jika akar sudah ada, proses dilanjutkan melalui fungsi rekursif *_insert_recursive* dengan membawa parameter node saat ini dan data baru yang akan dimasukkan. Dalam proses ini, nilai rating dari objek baru dibandingkan dengan rating pada node aktif.

Jika rating objek baru lebih kecil, maka proses akan diarahkan ke cabang kiri. Apabila node kiri masih kosong, maka node baru akan ditempatkan di posisi tersebut. Namun, jika sudah terisi, fungsi akan dipanggil kembali secara rekursif untuk menelusuri lebih dalam ke sisi kiri hingga ditemukan posisi yang sesuai. Sebaliknya, jika rating objek baru lebih besar atau sama, maka proses akan diarahkan ke cabang kanan dengan mekanisme yang sama hingga ditemukan posisi kosong.

Struktur penyusunan data yang terorganisir ini memberikan efisiensi tinggi dalam proses pencarian, khususnya melalui metode *search_exact_rating*. Metode ini bekerja dengan memanggil fungsi rekursif *_search_recursive* yang membandingkan nilai target dengan nilai pada setiap node yang dikunjungi. Jika node kosong atau nilai yang dicari ditemukan, maka proses dihentikan dan data dikembalikan. Jika nilai target lebih kecil, pencarian langsung diarahkan ke cabang kiri tanpa perlu memeriksa cabang kanan. Sebaliknya, jika nilai target lebih besar, pencarian dilanjutkan ke cabang kanan.

Pendekatan ini memungkinkan proses pencarian berjalan lebih cepat karena setiap langkah hanya menelusuri sebagian dari total data yang ada. Dengan demikian, kompleksitas waktu rata-rata yang dihasilkan adalah $O(\log N)$, yang jauh lebih efisien dibandingkan pencarian linier, karena sistem tidak perlu memeriksa seluruh elemen secara berurutan.

2.6 Algoritma Pencarian Sekuensial (Linear Search)

Berikut adalah implementasi fungsi filter data berbasis pencarian linear untuk mendeteksi kriteria anggaran belanja dan kapasitas memori:

```
108         return self._search_recursive(node.right, rating_target)
109
110     def linear_search_recommendation(hp_list, max_harga, min_ram):
111         hasil_filter = []
112         for hp in hp_list:
113             if hp.harga <= max_harga and hp.ram >= min_ram:
114                 hasil_filter.append(hp)
115         return hasil_filter
```

Penjelasan:

Algoritma pencarian linier atau *sequential search* pada program ini diimplementasikan melalui sebuah fungsi global bernama *linear_search_recommendation*. Fungsi ini berperan sebagai komponen utama dalam proses penyaringan data ketika pengguna

memasukkan kriteria pencarian pada sistem rekomendasi. Fungsi tersebut menerima tiga parameter, yaitu *hp_list* yang berisi kumpulan data handphone, *max_harga* sebagai batas maksimal anggaran pengguna, serta *min_ram* sebagai batas minimum kapasitas RAM yang diinginkan. Pada awal fungsi, dideklarasikan sebuah list kosong bernama *hasil_filter* yang digunakan untuk menampung data handphone yang memenuhi kriteria.

Proses kerja algoritma ini menggunakan perulangan *for* untuk menelusuri setiap elemen dalam *hp_list* secara berurutan, dimulai dari indeks pertama hingga terakhir. Pada setiap iterasi, satu objek handphone diambil dan diperiksa menggunakan kondisi logika. Pemeriksaan dilakukan dengan menggabungkan dua syarat menggunakan operator *and*, yaitu apakah harga handphone lebih kecil atau sama dengan batas anggaran, serta apakah kapasitas RAM lebih besar atau sama dengan batas minimum yang ditentukan.

Jika kedua kondisi tersebut terpenuhi, maka objek handphone dianggap sesuai dengan kebutuhan pengguna dan akan ditambahkan ke dalam list *hasil_filter*. Sebaliknya, jika salah satu atau kedua kondisi tidak terpenuhi, maka data tersebut akan dilewati dan proses dilanjutkan ke elemen berikutnya tanpa tindakan tambahan. Proses ini terus berlangsung hingga seluruh data dalam list selesai diperiksa.

Setelah seluruh proses penelusuran selesai, fungsi akan mengembalikan list *hasil_filter* yang berisi kumpulan handphone yang sesuai dengan kriteria pengguna. Hasil ini kemudian digunakan oleh sistem untuk ditampilkan pada antarmuka sebagai rekomendasi produk yang relevan.

2.7 Algoritma Pengurutan Selektif (Selection Sort)

Berikut adalah implementasi fungsi pengurutan data untuk menyusun peringkat nominal harga dari yang paling ekonomis:

```
116
117     def selection_sort_by_price(hp_list, ascending=True):
118         n = len(hp_list)
119         for i in range(n):
120             min_max_idx = i
121             for j in range(i + 1, n):
122                 if ascending:
123                     if hp_list[j].harga < hp_list[min_max_idx].harga:
124                         min_max_idx = j
125                 else:
126                     if hp_list[j].harga > hp_list[min_max_idx].harga:
127                         min_max_idx = j
128             hp_list[i], hp_list[min_max_idx] = hp_list[min_max_idx], hp_list[i]
129         return hp_list
130
```

Penjelasan:

Algoritma pengurutan data pada aplikasi ini menggunakan metode *Selection Sort* yang diimplementasikan dalam fungsi bernama *selection_sort_by_price*. Algoritma ini bertujuan untuk mengurutkan kumpulan objek handphone hasil proses penyaringan sebelumnya agar tersusun berdasarkan harga secara menaik, yaitu dari yang paling murah hingga paling mahal. Proses pengurutan dilakukan dengan memanfaatkan perulangan bersarang (*nested loop*) yang mengatur pergerakan indeks dalam list. Pada tahap awal, jumlah elemen dalam list dihitung menggunakan fungsi *len* dan disimpan dalam variabel *n*.

Perulangan luar dikendalikan oleh variabel indeks *i* yang bergerak dari posisi awal hingga akhir list. Pada setiap iterasi, diasumsikan bahwa elemen dengan nilai terkecil berada pada indeks *i*, yang kemudian disimpan dalam variabel *min_max_idx*. Selanjutnya, perulangan dalam dijalankan menggunakan variabel indeks *j*, yang dimulai dari posisi setelah *i* hingga akhir list untuk mencari elemen dengan harga paling rendah di sisa data yang belum terurut.

Dalam perulangan dalam, dilakukan proses perbandingan antara harga handphone pada indeks *j* dengan harga pada indeks yang ditunjuk oleh *min_max_idx*. Jika ditemukan harga yang lebih kecil, maka nilai *min_max_idx* akan diperbarui untuk menunjuk ke indeks tersebut. Proses ini berlangsung hingga seluruh elemen dalam bagian list yang belum terurut selesai diperiksa, sehingga posisi elemen dengan harga terendah berhasil ditemukan.

Setelah proses pencarian selesai, dilakukan pengecekan apakah posisi elemen minimum berbeda dengan posisi indeks *i*. Jika berbeda, maka dilakukan pertukaran posisi (*swap*) antara kedua elemen tersebut menggunakan mekanisme *tuple assignment* pada Python. Dengan demikian, elemen pada indeks *i* akan berisi handphone dengan harga paling rendah pada bagian tersebut.

Proses ini terus berlanjut pada iterasi berikutnya dengan mempersempit area data yang belum terurut. Setiap langkah memastikan bahwa elemen terkecil berikutnya ditempatkan pada posisi yang sesuai. Setelah seluruh proses selesai, list akan berada dalam kondisi terurut secara menaik berdasarkan harga, dan hasilnya kemudian dikembalikan untuk ditampilkan pada antarmuka pengguna.

2.8 Desain Antarmuka Grafis (Tkinter GUI Layout) dan Sinkronisasi Fungsi Operasional

Berikut adalah baris kode lengkap dari kelas utama yang menginisialisasi komponen visual jendela interaktif aplikasi desktop beserta integrasi fungsi manipulasi datanya:

```

131 class RecommendationApp:
132     def __init__(self, root):
133         self.root = root
134         self.root.title("Sistem Rekomendasi HP - Kelompok 5 (8 Materi)")
135         self.root.geometry("920x680")
136
137         self.database_hp = [
138             Handphone(1, "Infinix Note 40", 2500000, 8, 256, 4.5),
139             Handphone(2, "Samsung Galaxy A55", 5900000, 8, 256, 4.7),
140             Handphone(3, "Xiaomi Redmi Note 13", 2300000, 6, 128, 4.3),
141             Handphone(4, "iPhone 15 Pro", 19000000, 8, 128, 4.9),
142             Handphone(5, "Vivo V30", 4200000, 12, 512, 4.6)
143         ]
144         self.id_counter = 6
145         self.history_log = HistoryLinkedList()
146         self.compare_queue = ComparisonQueue()
147
148         self.create_widgets()
149         self.refresh_table(self.database_hp)
150
151     def create_widgets(self):
152         self.lbl_title = tk.Label(self.root, text="Daftar Handphone / Menu Utama CRUD", font=("Arial", 11, "bold"))
153         self.lbl_title.pack(pady=3)
154
155         table_frame = tk.Frame(self.root)
156         table_frame.pack(padx=10, fill=tk.X)
157
158         self.tree = ttk.Treeview(table_frame, columns=("ID", "Nama", "Harga", "RAM", "Storage", "Rating"), show="headings", height=5)
159         self.tree.pack(side=tk.LEFT, fill=tk.X, expand=True)
160         for col in ("ID", "Nama", "Harga", "RAM", "Storage", "Rating"):
161             self.tree.heading(col, text=col)
162             self.tree.column(col, width=100, anchor=tk.CENTER)
163
164         self.tree.bind("<<TreeviewSelect>>", self.on_row_select)
165         tk.Button(self.root, text="Tampilkan Semua Data", command=lambda: self.refresh_table(self.database_hp), bg="lightgray").pack(pady=2)
166
167         form_frame = tk.LabelFrame(self.root, text=" Form Manipulasi Data HP (CRUD) ")
168         form_frame.pack(padx=10, pady=2, fill=tk.X)
169
170         tk.Label(form_frame, text="Nama HP:").grid(row=0, column=0, padx=5, pady=2, sticky="e")
171         self.ent_nama = tk.Entry(form_frame, width=15)
172         self.ent_nama.grid(row=0, column=1, padx=5, pady=2)
173
174         tk.Label(form_frame, text="Harga (Rp):").grid(row=0, column=2, padx=5, pady=2, sticky="e")
175         self.ent_harga = tk.Entry(form_frame, width=15)
176         self.ent_harga.grid(row=0, column=3, padx=5, pady=2)
177
178         tk.Label(form_frame, text="RAM (GB):").grid(row=1, column=0, padx=5, pady=2, sticky="e")
179         self.ent_ram = tk.Entry(form_frame, width=15)
180         self.ent_ram.grid(row=1, column=1, padx=5, pady=2)
181
182         tk.Label(form_frame, text="Storage (GB):").grid(row=1, column=2, padx=5, pady=2, sticky="e")
183         self.ent_storage = tk.Entry(form_frame, width=15)
184         self.ent_storage.grid(row=1, column=3, padx=5, pady=2)
185
186         tk.Label(form_frame, text="Rating (0-5):").grid(row=2, column=0, padx=5, pady=2, sticky="e")
187         self.ent_rating = tk.Entry(form_frame, width=15)
188         self.ent_rating.grid(row=2, column=1, padx=5, pady=2)
189
190         crud_btn_frame = tk.Frame(form_frame)
191         crud_btn_frame.grid(row=3, column=0, columnspan=4, padx=5)
192         tk.Button(crud_btn_frame, text="Tambah HP (Create)", command=self.add_hp, bg="#a3e635").pack(side=tk.LEFT, padx=5)
193         tk.Button(crud_btn_frame, text="Simpan (Update)", command=self.update_hp, bg="#fde047").pack(side=tk.LEFT, padx=5)
194         tk.Button(crud_btn_frame, text="Hapus (Delete)", command=self.delete_hp, bg="#f87171", fg="white").pack(side=tk.LEFT, padx=5)
195
196         rec_frame = tk.LabelFrame(self.root, text="Sistem Rekomendasi (Linear Search & Selection Sort) ", fg="blue")
197         rec_frame.pack(padx=10, pady=2, fill=tk.X)
198
199         tk.Label(rec_frame, text="Max Budget:").grid(row=0, column=0, padx=5, pady=5)
200         self.ent_budget = tk.Entry(rec_frame, width=12)
201         self.ent_budget.grid(row=0, column=1, padx=5, pady=5)
202
203         tk.Label(rec_frame, text="Min RAM:").grid(row=0, column=2, padx=5, pady=5)
204         self.ent_min_ram = tk.Entry(rec_frame, width=8)
205         self.ent_min_ram.grid(row=0, column=3, padx=5, pady=5)
206
207         tk.Button(rec_frame, text="Cari Rekomendasi Termurah", command=self.get_recommendation, bg="#38bdf8").grid(row=0, column=4, padx=10, pady=5)
208

```

```

200: 14/10/2024 12:09:17 AD
Code Diame

209 bst_frame = tk.LabelFrame(self.root, text=" Pencarian Instan BST (Binary Search Tree) ", fg="purple")
210 bst_frame.pack(padx=10, pady=2, fill=tk.X)
211
212 tk.Label(bst_frame, text="Masukkan Target Rating (Contoh: 4.7):").pack(side=tk.LEFT, padx=10, pady=5)
213 self.ent_bst_rating = tk.Entry(bst_frame, width=8)
214 self.ent_bst_rating.pack(side=tk.LEFT, padx=5, pady=5)
215 tk.Button(bst_frame, text="Cari via BST Tree", command=self.search_bst, bg="#e9d5ff").pack(side=tk.LEFT, padx=10, pady=5)
216
217 queue_frame = tk.LabelFrame(self.root, text=" Fitur Bandingkan HP (Queue FIFO - Maksimal 3 Produk) ", fg="green")
218 queue_frame.pack(padx=10, pady=2, fill=tk.X)
219
220 tk.Button(queue_frame, text="Masukkan HP Terpilih ke Antrean Band", command=self.add_to_queue, bg="#b0f7d0").pack(side=tk.LEFT, padx=10, pady=5)
221 tk.Button(queue_frame, text="Keluarkan HP Tertua (Dequeue)", command=self.remove_from_queue, bg="#f2f2f2").pack(side=tk.LEFT, padx=10, pady=5)
222
223 self.lbl_queue_list = tk.Label(queue_frame, text="Antrean Saat Ini: (Kosong)", font=("Arial", 9, "italic"), fg="darkgreen")
224 self.lbl_queue_list.pack(side=tk.LEFT, padx=20)
225
226 nav_frame = tk.LabelFrame(self.root, text=" Log Riwayat Klik / Jelajah (Double Linked List) ")
227 nav_frame.pack(padx=10, pady=2, fill=tk.X)
228 self.lbl_history = tk.Label(nav_frame, text="Belum ada HP yang diklik.", font=("Arial", 9, "italic"), fg="gray")
229 self.lbl_history.pack(pady=2)
230
231 btn_container = tk.Frame(nav_frame)
232 btn_container.pack()
233 tk.Button(btn_container, text="◀ Riwayat Sebelumnya", command=self.prev_history).pack(side=tk.LEFT, padx=10, pady=2)
234 tk.Button(btn_container, text="Riwayat Selanjutnya ▶", command=self.next_history).pack(side=tk.LEFT, padx=10, pady=2)
235
236 def refresh_table(self, data_list):
237     for row in self.tree.get_children():
238         self.tree.delete(row)
239     for hp in data_list:
240         self.tree.insert("", tk.END, values=(hp.id, hp.nama, f"Rp(hp.harga:)", hp.ram, hp.storage, hp.rating))
241
242 def add_hp(self):
243     try:
244         nama = self.ent_nama.get()
245         harga = int(self.ent_harga.get())
246         ram = int(self.ent_ram.get())
247         storage = int(self.ent_storage.get())
248         rating = float(self.ent_rating.get())
249
250         ram = int(self.ent_ram.get())
251         storage = int(self.ent_storage.get())
252         rating = float(self.ent_rating.get())
253         if not nama: raise ValueError
254
255         new_hp = Handphone(self.id_counter, nama, harga, ram, storage, rating)
256         self.database_hp.append(new_hp)
257         self.id_counter += 1
258         self.refresh_table(self.database_hp)
259         messagebox.showinfo("Sukses", f"Berhasil menambahkan {nama}!")
260     except ValueError:
261         messagebox.showerror("Error", "Format input salah / belum lengkap!")
262
263 def on_row_select(self, event):
264     selected = self.tree.focus()
265     if not selected: return
266     values = self.tree.item(selected, 'values')
267
268     self.ent_nama.delete(0, tk.END)
269     self.ent_nama.insert(0, values[1])
270     self.ent_harga.delete(0, tk.END)
271     self.ent_harga.insert(0, values[2].replace("Rp", "").replace(", ", ""))
272     self.ent_ram.delete(0, tk.END)
273     self.ent_ram.insert(0, values[3])
274     self.ent_storage.delete(0, tk.END)
275     self.ent_storage.insert(0, values[4])
276     self.ent_rating.delete(0, tk.END)
277     self.ent_rating.insert(0, values[5])
278
279     id_hp = int(values[0])
280     for hp in self.database_hp:
281         if hp.id == id_hp:
282             self.history_log.add_history(hp)
283             self.lbl_history.config(text=f"Melihat: {hp.nama} [★{hp.rating}] ", fg="black")
284             break

```

```

281
282     def update_hp(self):
283         selected = self.tree.focus()
284         if not selected: return
285         values = self.tree.item(selected, 'values')
286         id_target = int(values[0])
287         try:
288             for hp in self.database_hp:
289                 if hp.id == id_target:
290                     hp.nama = self.ent_nama.get()
291                     hp.harga = int(self.ent_harga.get())
292                     hp.ram = int(self.ent_ram.get())
293                     hp.storage = int(self.ent_storage.get())
294                     hp.rating = float(self.ent_rating.get())
295                     break
296             self.refresh_table(self.database_hp)
297             messagebox.showinfo("Sukses", "Data HP diperbarui!")
298         except ValueError:
299             messagebox.showerror("Error", "Gagal memperbarui data!")
300
301     def delete_hp(self):
302         selected = self.tree.focus()
303         if not selected: return
304         values = self.tree.item(selected, 'values')
305         id_target = int(values[0])
306         self.database_hp = [hp for hp in self.database_hp if hp.id != id_target]
307         self.refresh_table(self.database_hp)
308         messagebox.showinfo("Sukses", "Data HP dihapus!")
309
310     def get_recommendation(self):
311         try:
312             budget = int(self.ent_budget.get())
313             min_ram = int(self.ent_min_ram.get())
314             # Searching & Sorting
315             filtered = linear_search_recommendation(self.database_hp, budget, min_ram)
316             sorted_res = selection_sort_by_price(filtered, ascending=True)
317             self.refresh_table(sorted_res)
318         except ValueError:
319             messagebox.showerror("Error", "Isi kriteria dengan angka!")
320

```



```

---
321     def search_bst(self):
322         try:
323             target = float(self.ent_bst_rating.get())
324             tree = HandphoneBST()
325             for hp in self.database_hp:
326                 tree.insert(hp)
327
328             hasil_hp = tree.search_exact_rating(target)
329             if hasil_hp:
330                 self.refresh_table([hasil_hp])
331                 messagebox.showinfo("BST Found", f"Ditemukan via BST:\n{hasil_hp}")
332             else:
333                 messagebox.showinfo("BST Not Found", "Tidak ada HP dengan rating bernilai tepat sebesar itu.")
334         except ValueError:
335             messagebox.showerror("Error", "Masukkan format angka desimal untuk rating BST!")
336
337     def add_to_queue(self):
338         selected = self.tree.focus()
339         if not selected:
340             messagebox.showwarning("Peringatan", "Pilih HP pada tabel terlebih dahulu!")
341             return
342         values = self.tree.item(selected, 'values')
343         id_hp = int(values[0])
344
345         for hp in self.database_hp:
346             if hp.id == id_hp:
347                 berhasil = self.compare_queue.enqueue(hp)
348                 if berhasil:
349                     self.update_queue_label()
350                 else:
351                     messagebox.showwarning("Antrean Penuh", "Maksimal membandingkan 3 HP. Keluarkan HP lama terlebih dahulu!")
352             break
353
354     def remove_from_queue(self):
355         item_keluar = self.compare_queue.dequeue()
356         if item_keluar:
357             messagebox.showinfo("Queue Dequeue", f"{item_keluar.nama} dikeluarkan dari list perbandingan.")
358             self.update_queue_label()
359         else:
360
361             else:
362                 messagebox.showinfo("Queue Kosong", "Tidak ada data HP di dalam antrean.")
363
364     def update_queue_label(self):
365         items = self.compare_queue.get_all()
366         if not items:
367             self.lbl_queue_list.config(text="Antrean Saat Ini: (Kosong)")
368         else:
369             teks_list = " vs ".join([hp.nama for hp in items])
370             self.lbl_queue_list.config(text=f"Bandingkan: [ {teks_list} ]")
371
372     def prev_history(self):
373         hp = self.history_log.get_previous()
374         if hp: self.lbl_history.config(text=f"Melihat (History): {hp.nama} [★{hp.rating}]")
375         else: messagebox.showinfo("Info", "Batas riwayat awal tercapai.")
376
377     def next_history(self):
378         hp = self.history_log.get_next()
379         if hp: self.lbl_history.config(text=f"Melihat (History): {hp.nama} [★{hp.rating}]")
380         else: messagebox.showinfo("Info", "Batas riwayat akhir tercapai.")
381
382 if __name__ == "__main__":
383     root = tk.Tk()
384     app = RecommendationApp(root)
385     root.mainloop()

```

```

|
if __name__ == "__main__":
    root = tk.Tk()
    app = RecommendationApp(root)
    root.mainloop()

```

Penjelasan:

Fungsi utama **create_widgets** dirancang untuk membangun struktur tata letak antarmuka grafis desktop secara sistematis dan berlapis dengan memanfaatkan pustaka *tkinter* serta *ttk*. Jendela utama aplikasi memiliki ukuran 920×680 piksel dan dibagi menjadi lima bagian utama yang berfungsi sebagai wadah komponen visual.

Pada bagian paling atas, terdapat komponen **Treeview** yang berperan sebagai tabel dinamis multi-kolom untuk menampilkan data seperti ID, nama handphone, harga, kapasitas RAM, penyimpanan, serta rating. Setiap kolom diatur dengan perataan tengah untuk menjaga kerapian tampilan. Selain itu, tabel ini juga dihubungkan dengan event *TreeviewSelect* yang memungkinkan sistem merespons aksi klik pengguna pada baris tertentu. Tepat di bawah tabel, disediakan sebuah tombol berwarna abu-abu yang berfungsi untuk mengembalikan tampilan data ke kondisi awal (reset).

Bagian kedua berupa **LabelFrame** yang digunakan sebagai form pengelolaan data (CRUD). Di dalamnya terdapat kombinasi label sebagai petunjuk dan entry sebagai tempat input data. Pada bagian bawah form ini terdapat tiga tombol aksi utama dengan warna berbeda untuk membedakan fungsi: tombol hijau untuk menambah data, tombol kuning untuk memperbarui data, dan tombol merah untuk menghapus data secara permanen dari memori.

Selanjutnya, bagian ketiga yang berwarna biru difungsikan sebagai area input untuk sistem rekomendasi berbasis linear search, khususnya untuk memasukkan budget dan RAM minimum. Bagian keempat berwarna ungu digunakan untuk fitur pencarian cepat berbasis struktur **Binary Search Tree (BST)**. Pada bagian paling bawah, terdapat dua kontainer tambahan yang berfungsi untuk mengatur antrean perbandingan produk (maksimal tiga item) serta navigasi riwayat penelusuran pengguna.

Fungsi **refresh_table** memiliki peran penting dalam menjaga konsistensi tampilan data pada layar. Saat fungsi ini dipanggil, langkah pertama adalah menghapus seluruh data lama dalam Treeview untuk menghindari duplikasi. Setelah itu, data baru dari list akan dimasukkan kembali ke dalam tabel dengan format harga yang telah disesuaikan menggunakan pemisah ribuan.

Proses penambahan data dilakukan oleh fungsi **add_hp** yang dilindungi oleh blok *try-except*. Fungsi ini mengambil input dari user, lalu mengonversinya ke tipe data yang sesuai (integer untuk harga, RAM, dan penyimpanan, serta float untuk rating). Jika

terjadi kesalahan input, sistem akan menampilkan pesan error tanpa menghentikan program. Jika data valid, objek baru akan ditambahkan ke dalam list database, ID akan diperbarui, dan tampilan tabel akan disegarkan.

Fungsi **on_row_select** bertugas menangani interaksi saat pengguna memilih baris pada tabel. Data dari baris tersebut akan otomatis dimasukkan ke dalam form input sehingga memudahkan proses edit. Selain itu, data tersebut juga disimpan ke dalam struktur **Double Linked List** sebagai riwayat penelusuran.

Fungsi **update_hp** digunakan untuk memperbarui data yang sudah ada. Sistem akan mencari data berdasarkan ID, kemudian mengganti seluruh atributnya dengan data baru dari form. Sementara itu, fungsi **delete_hp** menghapus data dengan cara membuat ulang list tanpa menyertakan data yang dipilih.

Pada fitur rekomendasi, fungsi **get_recommendation** mengambil input budget dan RAM minimum, lalu memprosesnya menggunakan algoritma **linear search** untuk menyaring data. Hasilnya kemudian diurutkan menggunakan **selection sort** berdasarkan harga termurah sebelum ditampilkan ke tabel.

Untuk pencarian berbasis BST, fungsi **search_bst** akan membangun struktur pohon dari data yang ada, kemudian mencari data berdasarkan nilai rating tertentu menggunakan metode pencarian pada BST.

Fitur antrean perbandingan dikendalikan oleh fungsi **add_to_queue** dan **remove_from_queue**. Data yang dipilih dapat dimasukkan ke dalam antrean maksimal tiga item. Jika antrean penuh, sistem akan memberikan peringatan. Saat data dikeluarkan dari antrean, sistem akan menampilkan informasi produk yang dihapus dan memperbarui tampilan antrean.

Navigasi riwayat penelusuran diatur oleh fungsi **prev_history** dan **next_history**, yang memungkinkan pengguna melihat data sebelumnya atau berikutnya berdasarkan urutan interaksi.

Seluruh proses ini diakhiri dengan pemanggilan **mainloop**, yang menjaga aplikasi tetap berjalan dan responsif terhadap interaksi pengguna.

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, serta pengujian yang telah dilakukan pada Sistem Rekomendasi Handphone, dapat diambil beberapa kesimpulan sebagai berikut:

1. Penerapan konsep Pemrograman Berorientasi Objek (OOP) melalui pembuatan kelas *Handphone* terbukti efektif dalam mengelola data dengan berbagai atribut. Dengan adanya enkapsulasi, data dapat tersusun rapi dalam satu objek sehingga memudahkan pengelolaan dan mengurangi kesalahan.
2. Penggunaan struktur data list sebagai media penyimpanan utama memberikan fleksibilitas dalam pengolahan data. Sistem dapat menjalankan operasi CRUD (tambah, lihat, ubah, hapus) dengan mudah serta mampu menampilkan data secara konsisten pada antarmuka.
3. Implementasi Double Linked List berhasil digunakan sebagai sistem pencatatan riwayat aktivitas. Struktur ini memungkinkan pengguna untuk menelusuri data sebelumnya maupun berikutnya secara efisien.
4. Struktur data queue dengan konsep FIFO berhasil diterapkan pada fitur perbandingan produk. Dengan adanya batas maksimal antrean, sistem tetap terkontrol dan mekanisme FIFO memastikan data diproses sesuai urutan masuk.
5. Penggunaan Binary Search Tree (BST) dalam pencarian data berdasarkan rating mampu meningkatkan efisiensi, karena proses pencarian tidak perlu dilakukan secara berurutan pada seluruh data.
6. Penggabungan algoritma linear search dan selection sort dapat menghasilkan rekomendasi produk yang optimal. Linear search menyaring data sesuai kriteria, sedangkan selection sort mengurutkan berdasarkan harga termurah.
7. Penggunaan antarmuka grafis berbasis Tkinter membuat sistem lebih interaktif dan mudah digunakan. Selain itu, penerapan error handling dengan try-except membantu menjaga stabilitas program.

Secara keseluruhan, sistem yang dibangun telah berjalan dengan baik dan mampu membantu pengguna dalam memilih handphone secara lebih cepat, tepat, dan efisien.